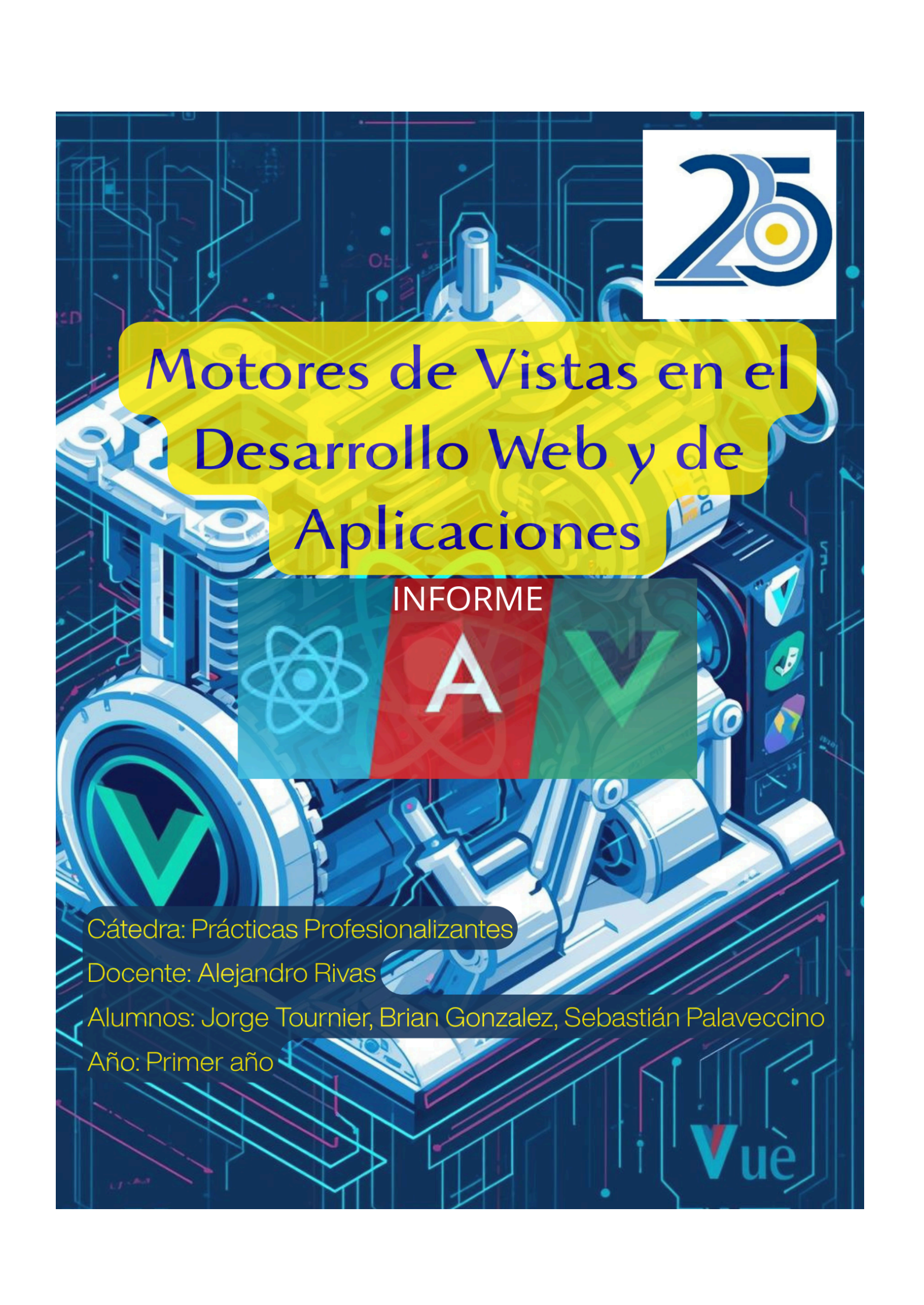




25

Motores de Vistas en el Desarrollo Web y de Aplicaciones

INFORME



A



Cátedra: Prácticas Profesionalizantes

Docente: Alejandro Rivas

Alumnos: Jorge Tournier, Brian Gonzalez, Sebastián Palavecino

Año: Primer año

Vuè

1. Introducción

El desarrollo web moderno se caracteriza por la necesidad de construir aplicaciones cada vez más dinámicas, rápidas y fáciles de usar. En sus orígenes, la web estaba formada por páginas estáticas escritas únicamente en HTML, pero hoy los usuarios esperan interfaces interactivas que respondan en tiempo real a sus acciones.

En este contexto surgen los motores de vistas, también conocidos como frameworks o bibliotecas de frontend, que permiten gestionar cómo se construyen y actualizan las interfaces gráficas. Herramientas como React, Angular y Vue.js se han convertido en pilares fundamentales de la industria, ya que simplifican el trabajo de los desarrolladores y garantizan experiencias de usuario más fluidas.

Este informe busca explicar qué son los motores de vistas, cómo funcionan y analizar los más utilizados en la actualidad: React, Angular, Vue, Razor (C#) y los motores en Python.

2. ¿Qué es un motor de vistas?

Un motor de vistas es la tecnología que se encarga de transformar datos en representaciones visuales dinámicas, es decir, convierte la información gestionada por el backend en elementos que el usuario puede ver e interactuar en el frontend.

- **Backend:** gestiona la lógica de negocio y las bases de datos.
- **Frontend:** muestra esa información en una interfaz amigable.
- **Motor de vistas:** conecta ambos mundos, renderizando la información y actualizándola automáticamente cuando cambian los datos.

Características comunes de un motor de vistas:

- **Plantillas dinámicas:** HTML enriquecido con variables.
 - **Renderizado eficiente:** se actualizan solo las partes necesarias, no toda la página.
 - **Componentización:** la UI se divide en bloques reutilizables.
 - **Interactividad:** responde a eventos del usuario como clics o formularios.
-

3. Motores de vistas modernos en JavaScript

3.1 React

- **Origen:** Facebook (2013)
- **Tipo:** Biblioteca de JavaScript
- **Características:**
 - JSX (HTML + JS en un mismo archivo)
 - Virtual DOM para actualizaciones rápidas
 - Flujo de datos unidireccional
 - Ecosistema enorme (Redux, React Router, etc.)

Ejemplo:

```
function Saludo(props) {  
  return <h1>Hola, {props.nombre}</h1>;  
}
```

3.2 Angular

- **Origen:** Google (2010 como AngularJS, reescrito en 2016)
- **Tipo:** Framework completo (SPA)
- **Lenguaje:** TypeScript
- **Características:**
 - Arquitectura MVC
 - Data binding bidireccional
 - CLI oficial muy robusta
 - Ideal para aplicaciones grandes y corporativas

Ejemplo:

```
<h1>Hola, {{ nombre }}</h1>
```

3.3 Vue.js

- **Origen:** Evan You (2014)
- **Tipo:** Framework progresivo
- **Características:**
 - Sintaxis clara y fácil
 - Virtual DOM como React
 - Reactividad automática
 - Ecosistema propio (Vue Router, Vuex)

Ejemplo:

```
<div id="app">  
  <h1>Hola, {{ nombre }}</h1>  
</div>
```

```
<script>  
new Vue({  
  el: '#app',  
  data: { nombre: 'Lucía' }  
})  
</script>
```

4. Motores de vistas en otros ecosistemas

4.1 Razor (C# / ASP.NET Core)

- Motor de vistas oficial de Microsoft
- Combina HTML con C# en archivos `.cshtml`
- Usado en ASP.NET Core MVC y Blazor

Ejemplo:

```
<h1>Hola, @Model.Nombre</h1>
```

4.2 Motores de vistas en Python

- **Django Templates (DTL)**: motor oficial de Django, seguro y estructurado
- **Jinja2 (Flask)**: muy popular, sintaxis parecida a Django
- Otros: Mako, Chameleon, Tornado Templates

Ejemplo en Jinja2:

```
<ul>

{% for producto in productos %}

    <li>{{ producto.nombre }} - {{ producto.precio }}</li>

{% endfor %}

</ul>
```

5. Comparación general

**Motor /
Framework**

**Lenguaje
principal**

**Data
Binding**

Filosofía

Uso ideal

React	JavaScript / TS	Unidireccional	Virtual DOM, componentes	Interfaces dinámicas
Angular	TypeScript	Bidireccional	Framework completo, MVC	Grandes proyectos
Vue.js	JavaScript / TS	Bidireccional	Progresivo, flexible	Proyectos medianos
Razor	C# + HTML	—	Plantillas servidor	Web con ASP.NET
Django/Jinja2	Python + HTML	—	Plantillas servidor	Web con Python

6. Aplicaciones móviles y multiplataforma

- **React Native (JS/TS):** apps móviles y de TV
- **.NET MAUI (C# + XAML):** desarrollo móvil y escritorio con un solo código
- **Blazor Hybrid (C# + Razor):** integra Blazor en apps móviles

7. Flujo de datos: unidireccional vs bidireccional

7.1 Flujo unidireccional (one-way data binding)

- Los datos fluyen solo en una dirección: **Modelo** → **Vista**
- Cambios en la vista no alteran el modelo directamente, requieren eventos

Ejemplo en React:

```
function App() {  
  const [nombre, setNombre] = React.useState("Lucía");  
  return (  
    <div>  
      <h1>Hola, {nombre}</h1>  
      <input  
        value={nombre}  
        onChange={e => setNombre(e.target.value)}  
      />  
    </div>  
  );  
}
```

Ventajas:

- Más predecible
- Menos errores al escalar proyectos grandes

7.2 Flujo bidireccional (two-way data binding)

- Los datos fluyen en ambas direcciones: **Modelo** ↔ **Vista**
- Cambios en la vista actualizan el modelo automáticamente

Ejemplo en Angular:

```
<input [(ngModel)]="nombre">  
  
<p>Hola, {{ nombre }}</p>
```

Ventajas:

- Más rápido y sencillo de escribir
- Ideal para formularios o interfaces simples

Desventaja:

- Puede volverse difícil de rastrear en proyectos grandes

Resumen comparativo:

Característica	Unidireccional	Bidireccional
Flujo de datos	Modelo → Vista	Modelo ↔ Vista
Ejemplo	React, Vue (modo controlado)	Angular, Vue (con v-model)
Ventaja	Más control y claridad	Más simple de usar
Desventaja	Requiere más código	Puede ser confuso en apps grandes

8. Cronología de los motores de vistas y frameworks frontend

8.1 Era del HTML + JS plano (1990 – 2005)

- Páginas web estáticas, manipulación imperativa del DOM
- Problema: difícil de mantener y escalar
- Motivo de evolución: dinamismo sin recargar toda la página

8.2 Aparición de librerías de soporte (2005 – 2010)

- **jQuery (2006):** simplifica DOM, animaciones y AJAX
- Motivo de evolución: manejar aplicaciones más grandes

8.3 Primeros frameworks MVC en frontend (2010 – 2013)

- **Backbone.js (2010):** estructura MVC básica
- **AngularJS (2010):** binding bidireccional y plantillas declarativas

8.4 React y el modelo unidireccional (2013 – 2015)

- Virtual DOM, JSX y flujo unidireccional
- Motivo: escalabilidad y rendimiento

8.5 Competencia y refinamiento (2014 – hoy)

- **Vue.js (2014):** combina Angular y React, binding opcional
- **Svelte (2019):** compila componentes a JS puro para optimizar rendimiento

8.6 Tendencias actuales

- React: lidera en proyectos grandes
- Vue: popular en startups y comunidades independientes
- Svelte, Solid.js, Qwik: nuevas ideas de rendimiento y renderizado

Resumen de evolución en línea de tiempo:

Año	Tecnología	Filosofía principal	Motivo de aparición
1990s	HTML + JS	Imperativo, recarga total	Sitios estáticos
2006	jQuery	Simplificar DOM y AJAX	Web más interactiva

2010	Backbone.js	MVC ligero	Estructurar apps grandes
2010	AngularJS	Binding bidireccional + plantillas	Automatizar sincronía vista-modelo
2013	React	Virtual DOM + unidireccional	Escalabilidad y rendimiento
2014	Vue.js	Reactividad fácil + plantillas	Combinar simplicidad y potencia
2019	Svelte	Compilación a JS puro	Optimizar rendimiento

9. Aplicaciones importantes según el motor de vistas

9.1 React

- Facebook, Instagram, WhatsApp Web, Netflix, Airbnb, Dropbox

9.2 Angular

- Google Cloud Console, Microsoft Office Online, Deutsche Bank, Forbes, Gmail

9.3 Vue.js

- Alibaba, Xiaomi, Nintendo, GitLab, Behance (Adobe)

9.4 Razor (C# / ASP.NET)

- Stack Overflow, Microsoft Docs, GoDaddy, DHL, sitios gubernamentales

9.5 Python (Django/Jinja2)

- Instagram (origen), Pinterest, Disqus, Mozilla, The Washington Post

Conclusión:

- React → apps masivas y plataformas de streaming
 - Angular → banca, gobierno y corporaciones grandes
 - Vue.js → startups y productos de iteración rápida
 - Razor → entornos corporativos Microsoft
 - Python → medios digitales e investigación
-

10. Conclusiones

Los motores de vistas han transformado el desarrollo de software al permitir construir aplicaciones más dinámicas, interactivas y mantenibles.

- **React:** flexibilidad y comunidad
- **Angular:** robustez y entornos corporativos
- **Vue:** simplicidad y curva de aprendizaje baja
- **Razor / Python:** entornos donde C# o Python dominan el backend

Elección según contexto:

- Escalabilidad y robustez → Angular
- Flexibilidad y ecosistema → React
- Simplicidad y progresividad → Vue
- Integración con entornos empresariales → Razor o Django/Jinja2